Implementation and Analysis of Advanced Navigation Algorithms for Robotic Vacuum Cleaners Using Webots Simulator

A Keerthana Chowdary
School Of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
cb.ai.u4aid23001@cb.students.amrita.edu

Kota KedharNadh
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
cb.ai.u4aid23015@cb.students.amrita.edu

M SaiKrishna
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
cb.ai.u4aid23020@cb.students.amrita.edu

M Rishitha
School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
cb.ai.u4aid23023@cb.students.amrita.edu

Akhil V.M
school of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
vm akhil@cb.amrita.edu

***Abstract*—Robotic vacuum cleaners have increasingly become a staple in modern households, providing an automated and efficient means of maintaining cleanliness with minimal human oversight. The core of these robots' functionality lies in their navigation and obstacle avoidance systems, which are driven by sophisticated algorithms that enable them to move efficiently within a space, navigate around obstacles, and optimize their cleaning coverage. This paper conducts a comprehensive analysis of several advanced navigation algorithms implemented on a robotic vacuum cleaner model within the Webots simulation environment. The algorithms explored in this study include Breadth-First Search (BFS) Cleaning, Simultaneous Localization and Mapping (SLAM) combined with a Zigzag Pattern, Boustrophedon Motion, Wavefront Planning with Potential Field Method, A* Path Planning, Probabilistic Harmonic Functions (PHF), Rapidly-Exploring Random Trees (RRT), and Random Walk. The performance of these algorithms is rigorously evaluated in terms of total area cleaned, path efficiency, and obstacle avoidance, using a fixed period of 40 minutes as the benchmark. The findings of this study reveal essential trade-offs between algorithm complexity, computational overhead, and overall cleaning effectiveness. These insights provide a roadmap for future advancements in robotic vacuum cleaner technology, potentially paving the way for more intelligent, efficient, and adaptive cleaning systems.**

***Index Terms*—Robotic Vacuum Cleaners, Navigation Algorithms, Webots Simulation, SLAM, Path Planning.**

I. Introduction

Robotic vacuum cleaners have revolutionized home cleaning by providing autonomous, efficient, and time-saving solutions. Central to their functionality are the navigation systems, that allow these robots to move intelligently, detect and avoid obstacles, and systematically clean large areas without any human input. The success of these systems heavily depends on the sophistication of their navigation algorithms, which are designed to map unfamiliar environments, optimize movement, and handle dynamic obstacles[1]. The rapid advancements in sensor technology, machine learning, and computational methods have paved the way for a variety of navigation algorithms used in these devices. While simpler methods, such as random walk algorithms, are easy to implement, they often fall short in terms of efficiency and coverage. More complex strategies, including SLAM (Simultaneous Localization and Mapping) and A* Path Planning, offer significant improvements but require greater computational resources and advanced hardware[2]. The development of smart home appliances has changed the way we live. Especially the development of the smart vacuum cleaner has seen significant growth in the industry. The recent advancements not only focus on basic cleaning but also introduce obstacle avoidance, autonomous navigation, Bluetooth-based connection, decrease in power consumption. They highlighted the use of ultrasonic sensors for obstacle detection which decreases the chances of collisions and increases the performance of navigation[3]. P. B. Jarande and S. P. Murakar designed the robot integrating the Arduino and wifi connection which improves the user experience. Integrating wifi here increases the range when compared to Bluetooth-based robots and they also highlight the importance of obstacle detection[4]. The existing vacuum robots cannot clean the staircases. So to overcome this limitation, integrating Arduino Mega 2560 with the path planning algorithm Boustrophedon is very efficient. This integration not only helped in cleaning the staircases but also in different levels of the floors and types of floors[5]. The IoT-enabled vacuum cleaners also have improved technology that helps people to control the robot to clean in different areas by simply handling the control of the robot through our mobile phones which also increases the efficiency in the household tasks[6]. During the pandemic, we came to know the importance of cleanliness and being

sterilized. So integrating the sterilization part into the smart vacuum cleaners is the core concept. Here the robots are integrated with a removable sterilizing device which is quite useful in hospitals[7]. There are recent research studies where the CNN is used for feature extraction and accuracy analysis which can be helpful in real-time obstacle detection that can enhance the navigation in the vacuum robots[8].

This paper focuses on examining a spectrum of navigation algorithms, each with varying levels of complexity, within the Webots simulation environment. Webots is an open-source, highly versatile robotic simulation platform that supports the realistic modeling of robots and their environments. By leveraging this tool, we can perform extensive testing and fine-tuning in a virtual setup, circumventing the financial and logistical constraints of physical prototyping. The simulation environment also allows for the creation of realistic scenarios, complete with obstacles and room layouts that mimic typical household settings.

The objective of this paper is to evaluate these algorithms comprehensively, assessing how effectively they perform in simulated household environments with static obstacles. Key performance metrics include the area covered by the robot, the distance traveled, and the robot's capability to avoid obstacles. Additionally, the study examines the balance between algorithm complexity and computational resource requirements, with an emphasis on understanding which approaches are most practical and efficient for real-world applications. Methodology explains the implementation and navigation algorithms using webots simulator. Algorithms Development And Implementation provides the explanations of the different algorithms used with the essesntial mathematics required. PID Controller Implementation section explains about the PID controller how it helps to get the stability and tracking of the vacuum cleaner. In the Results and Discussion section we discussed what the results are for the simulation we did for the diffferent algorithms.

II. Methodology

The navigation algorithms were developed and implemented using Python in the Webots platform to evaluate their effectiveness in room coverage and obstacle avoidance. Integration with Webots was done by programming the algorithms in Python to utilize Webots' robot control functions. This enabled seamless interaction with sensors and actuators modeled on the virtual robotic vacuum cleaner. Parameter optimization was crucial; parameters such as movement speed, sampling intervals, and sensor range were fine-tuned to enhance performance and ensure smooth operation within the simulation environment.

Webots, a very useful platform simulation, provided a comprehensive environment for modeling, programming, and simulating mobile robots. It offered realistic physics simulations, including dynamics, kinematics, and sensor feedback, which made it an ideal platform for testing and analyzing navigation algorithms.

The robotic vacuum cleaner model used in the simulations had specific features. It was 0.35 meters in diameter and 0.1 meters in height. It was equipped with infrared distance sensors for collision detection, LIDAR for mapping, bump sensors used for obstacle detection when it physically detects any obstacle, and wheel encoders for movement tracking. The differential drive system allowed independent wheel control, which was essential for flexible and efficient navigation. The simulated environment was set up as a 10m x 10m room, replicating a typical household space. This room has static obstacles, such as furniture, occupying around 15% of the total area. The floor was designed to be flat and uniform, allowing the robot to focus on efficient cleaning and obstacle avoidance.

III. Algorithms Development and Implementation

A. Breadth-First Search

BFS Cleaning was used to create a grid-based map where the robot systematically explored all reachable cells in a breadth-first manner, ensuring complete coverage of the area. The environment was discretized into a grid where BFS explored by initializing a queue with the starting cell, marking visited cells, and enqueuing unvisited adjacent cells. Implementation in Webots involved dividing the environment into 0.5-meter cells and using PID-controlled wheel motors for robot movement. This method ensured full coverage but was inefficient due to potential backtracking.

The environment is discretized into a grid $G = (V, E)$, where V represents the set of cells, and E represents the edges between adjacent cells. BFS explores the grid by initializing a queue Q with the starting cell v_s,Marking each visited cell to prevent re-exploration and Enqueuing all unvisited adjacent cells of the current cell.

The time complexity is $O(V + E)$.

B. SLAM with Zig-Zag pattern

It enables the robot to build a map while tracking its location and covers the area in a zigzag motion. The SLAM equations estimated the robot's state and map using probabilistic models. The robot moved straight until reaching a boundary or obstacle and then shifted laterally. LIDAR data updated the occupancy grid map, while wheel encoders and sensor data estimated the robot's position. This approach offered efficient coverage but required significant computational resources.

SLAM Equations: The robot's state x_t and the map m_t are estimated using probabilistic models:

$$p(x_t, m_t \mid z_{1:t}, u_{1:t}) = p(x_t \mid x_{t-1}, u_t) \cdot p(z_t \mid x_t, m_{t-1}) \quad (1)$$

where z_t is known as observation at time t, and u_t is the control input.

C. Boustrophendon Motion

It divides the environment into cells, covering each in a back-and-forth pattern. The area was decomposed into simple cells, and the robot followed a planned path within each cell, using PID controllers for precise movement. This method was efficient in structured environments but became complex with irregular shapes.

Cell Decomposition:

$$\text{Area} = \bigcup_{i=1}^{n} \text{Cell}_i \tag{2}$$

where Cell_i are non-overlapping regions.

D. Wavefront Planning with Potential Field Method

It creates a cost map and used virtual forces to guide the robot. The cost map was generated through wavefront propagation, and virtual forces were computed from the potential field gradients. The robot moved in the direction of the resultant force. This method provided smooth navigation but could suffer from local minima issues.

Wavefront Planning: The cost of each cell is calculated using:

$$C(i,j) = \min\{C_{\text{neighbors}}\} + 1 \tag{3}$$

starting from the goal cell with $C_{\text{goal}} = 0$.

Potential Field Method: The total potential U is given by:

$$U(\mathbf{q}) = U_{\text{att}}(\mathbf{q}) + U_{\text{rep}}(\mathbf{q}) \tag{4}$$

where U_{att} is the attractive potential towards the goal, and U_{rep} is the repulsive potential from obstacles.

E. A* Path Planning

It is a path planning algorithm that finds the shortest path which minimizes the cost of the working. The environment was designed as a grid graph, and the heuristic was the Euclidean distance. This method guarantees the shortest path but requires complete knowledge of the environment and has high memory usage.

The cost function $f(n)$ of A* path planning:

$$f(n) = g(n) + h(n) \tag{5}$$

where:

- $g(n)$ is known as the cost from the start node to node n.
- $h(n)$ is known as the heuristic estimate from the node n to the goal.

F. Probablisitic Harmonic Funcion

PHF employed harmonic functions to model smooth paths that inherently avoided obstacles. The potential field is computed by solving Laplace's equation numerically, with the robot navigating using the negative gradient of the field. This method produced smooth paths but was computationally expensive.

Harmonic functions satisfy Laplace's equation:

$$\nabla^2 U(\mathbf{q}) = 0 \tag{6}$$

Boundary conditions are set at obstacles and goals to compute the potential field.

G. Rapidly-Exploring Random Trees (RRT)

RRT rapidly explored high-dimensional spaces by building a random tree, sampling points, finding the nearest node, and extending the tree iteratively. The sampling space was defined by the occupancy grid, with collision checking to ensure paths did not intersect obstacles. The robot navigated through the generated tree nodes, which worked well in complex environments but could result in non-optimal paths. The algorithm iteratively builds a tree T by:

- Sampling a random point q_{rand}.
- Finding the nearest node q_{near} in T.
- Extending from q_{near} towards q_{rand} by a step size δq.

H. Random Walk

The Random Walk algorithm moved the robot in random directions, changing course upon collision. Each step involved selecting a random direction and moving forward. While simple to implement, this method was inefficient, often resulting in repeated paths and low coverage.

I. CNN model

CNN model is employed in this study for dirt detection in the Webots robotic vacuum cleaner simulation. Using the camera in the simulated environment, images of surfaces of different levels of cleanness were taken. The images gathered are labeled manually into dirt presence(Fig 2) and absence of dirt(Fig 1). We trained the model based on the dataset we have gathered. So when the robot starts moving the camera takes the image and with the help of the trained model the robot knows whether that area should be cleaned or not if the area is cleaned it will skip that area and if the area is not cleaned it will go to that area and start cleaning, like the cleaning goes on till the whole room is covered and it is cleaned.

IV. Results and Discussion

The simulations used a time step of 64 milliseconds, with a duration of 40 minutes per algorithm. The obstacle density in the room was about 15%, and the robot's maximum speed was 0.5 meters per second. Performance metrics included coverage percentage, total area cleaned, and path length. Algorithms like SLAM with Zigzag and Wavefront with Potential Fields achieved high coverage, while Random Walk was inefficient. RRT and PHF were computationally demanding, affecting real-time performance. Table 1 gives information about the performance metrics of the algorithms that we evaluated for a period of 40 minutes.

There were trade-offs between algorithm complexity and performance. Advanced algorithms offered higher coverage and efficiency but demanded more computational power. Fig 4 shows the area covered over a period of time for the different Navigation algorithms. However, the simulation had limitations, such as simplified real-world factors like battery life. The CNN model's accuracy for both the training and validation is depicted in Fig 3. The training accuracy of the model is 90% by the end of the training and the validation is about 80%. The fluctuations in the graph of validation

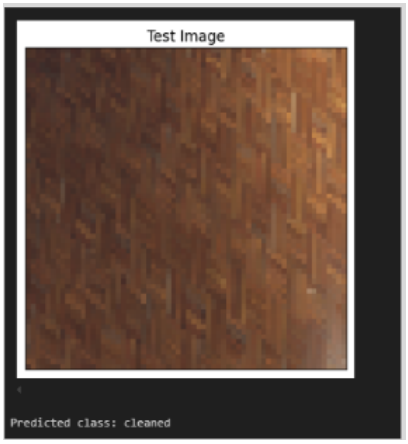


Fig. 1. Clean area

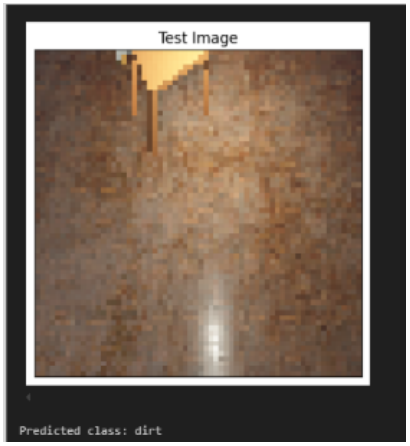


Fig. 2. Dirt is detected

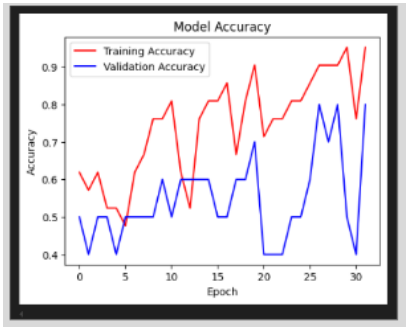


Fig. 3. CNN Model accuracy

TABLE I
PERFORMANCE METRICS OF ALGORITHMS OVER 40 MINUTES

Algorithm	Coverage (%)	Time (min)
BFS Cleaning	82	40
SLAM + Zigzag	85	40
Boustrophedon	83	40
Wavefront + PF	78	40
A* Planning	79	40
PHF Method	80	40
RRT	82	40
Random Walk	72	40

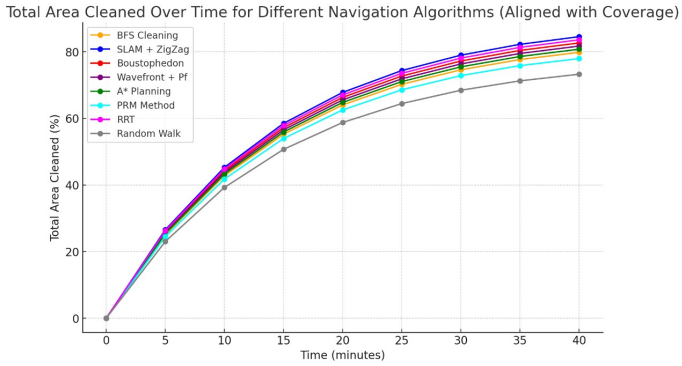


Fig. 4. Total Area Cleaned Over Time for Different Navigation Algorithms (Revisiting Areas Midway)

accuracy show that there might be chances of overfitting which means the model performs well with the known data and poor with the unseen data.

V. CONCLUSION

This paper compared the eight navigation algorithms for robotic vacuum cleaners in Webots, SLAM combined with the Zigzag Pattern achieved the best coverage but required more computational resources, and Random Walk has the poor coverage of the room among all the other algorithms though it is not computationally very less expensive. Boustrophedon is the next best algorithm after the SLAM combined with Zigzag Pattern that has a better accuracy. Future work could be testing these algorithms on physical robots, adding dynamic obstacles, developing hybrid algorithms, and optimizing energy consumption.

VI. REFERENCES

[1]M. I. Hasan *et al.*, "Scheme and Construction of a Smart Vacuum Cleaner Robot," *2023 Third International Conference on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT)*, Bhilai, India, 2023, pp. 1-6, doi: 10.1109/ICAECT57570.2023.10118212

[2]B. Liu, Z. Guan, B. Li, G. Wen and Y. Zhao, "Research on SLAM Algorithm and Navigation of Mobile Robot Based on ROS," *2021 IEEE International Conference on Mechatronics and Automation (ICMA)*, Takamatsu, Japan, 2021, pp. 119-124, doi: 10.1109/ICMA52036.2021.9512584

[3]L. Giri, A. R. Qureshi, B. K. Chethan, J. Parveen F, and M. S. Neha, ”Smart vacuum cleaner using Bluetooth module,” *International Research Journal of Modernization in Engineering, Technology and Science*, vol. 5, no. 6, 2023. doi: 10.56726/IRJMETS42722.

[4] P. B. Jarande, S. P. Murakar, N. S. Vast, N. P. Ubale and S. S. Saraf, ”Robotic Vacuum Cleaner Using Arduino with Wifi,” *2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*, Coimbatore, India, 2018, pp. 1513-1517, doi: 10.1109/ICICCT.2018.8473256.

[5]G. G. Gavindra, H. Kusuma and Tasripan, ”Vacuum Cleaner Robot with Staircase Cleaning Feature and Boustrophedon Path Planning,” *2021 International Seminar on Intelligent Technology and Its Applications (ISITIA)*, Surabaya, Indonesia, 2021, pp. 359-363, doi: 10.1109/ISITIA52817.2021.9502216

[6]S. S. Shariffudin, M. B. Abdul Razak, P. S. Mohamad Saad, H. Hashim and M. H. Mamat, ”IOT-Enabled Vacuum Cleaner Using Arduino,” *2023 IEEE Symposium on Industrial Electronics & Applications (ISIEA)*, Kuala Lumpur, Malaysia, 2023, pp. 1-6, doi: 10.1109/ISIEA58478.2023.10212381.

[7]A. Mohan and A. R. Krishnan, ”Design and Simulation of an Autonomous Floor Cleaning Robot with Optional UV Sterilization,” *2022 IEEE 2nd Mysore Sub Section International Conference (MysuruCon)*, Mysuru, India, 2022, pp. 1-6, doi: 10.1109/MysuruCon55714.2022.9972558.

[8]S. H. Bala, K. P. Prajith, B. Siddharth, and B. N. Prashanth, ”Design and Analysis of Automatic Robotic Vacuum Cleaner,” *International Journal of Advanced Research in Engineering and Technology (IJARET)*, vol. 11, no. 11, pp. 728-736, Nov. 2020, doi: 10.34218/IJARET.11.11.2020.069

[9]H. Lee and A. Banerjee, ”Intelligent scheduling and motion control for household vacuum cleaning robot system using simulation based optimization,” *2015 Winter Simulation Conference (WSC)*, Huntington Beach, CA, USA, 2015, pp. 1163-1171, doi: 10.1109/WSC.2015.7408242.

[10]Kavita, R. Kashyap, D. Kumar, D. Kumar, and P. Jaiswal, ”A Review Paper on ’Smart Vacuum Cleaner’,” in *Proceedings of the Department of Electronics and Communication Engineering*, Goel Institute of Technology and Management, Lucknow, India, 2023.

[11][1] S. Patil, S. Yelmar, S. Yedekar, S. Mhatre, and V. Pawashe, ”Autonomous Robotic Vacuum Cleaner,” in *International Research Journal of Innovations in Engineering and Technology (IRJIET)*, pp. 142-146, 2021. doi: 10.47001/IRJIET/2021.504021.